BIBLIOTECA VIRTUAL

Documentação Técnica

Projeto Web em Node.js, Express e MySQL

Projeto	**Biblioteca Virtual**
Versão do documento	1.0 **estável**
Data	18 de setembro de 2026
Stack principal	Node.js · Express · EJS · Bootstrap 5 · MySQL (Knex)
Servidor padrão	`http://localhost:3000`

Sumário

1. Introdução e objetivos
2. Visão geral da arquitetura
3. Tecnologias e dependências
4. Estrutura de diretórios
5. Arquitetura em camadas (MVC)
6. Modelo de dados (banco de dados)
7. Fluxo de requisições
8. Rotas da aplicação
9. Camada de visão (templates EJS)
10. Fluxograma geral de uma requisição
11. Como executar o projeto
12. Pontos de evolução

Seção 1

Introdução e objetivos

A **Biblioteca Virtual** é uma aplicação web para gerenciamento de livros. A interface atual permite *listar* o acervo cadastrado e *cadastrar* novos títulos, persistindo-os em um banco de dados MySQL. As páginas são servidas de forma dinâmica pelo servidor (server-side rendering) usando o motor de templates EJS.

Os objetivos principais desta documentação são:

- Descrever a arquitetura e o papel de cada módulo do sistema;
- Documentar o modelo de dados e o ciclo de vida de uma requisição;
- Servir de referência para manutenção e evolução do código.

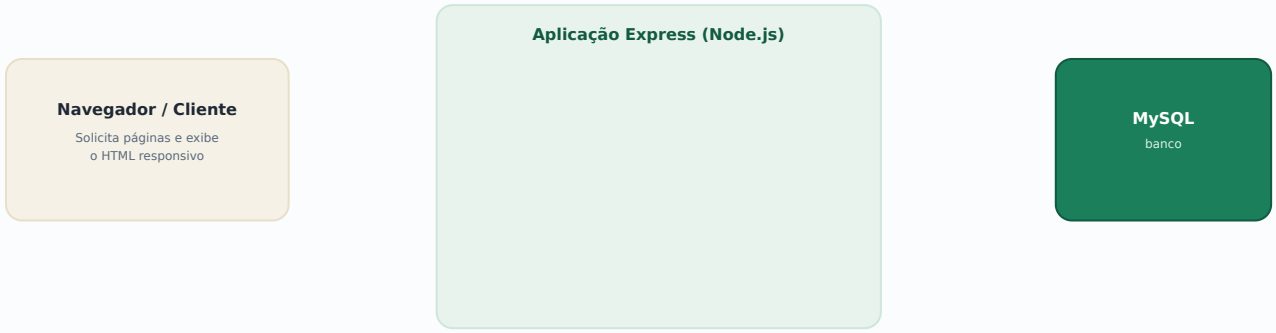
Funcionalidades atualmente implementadas

#	Funcionalidade	Rota	Situação
1	Listar todos os livros (tela inicial)	`GET /`	Implementado
2	Exibir formulário de cadastro	`GET /books/new`	Implementado
3	Cadastrar um novo livro	`POST /books/new`	Implementado
4	Exibir, editar e excluir livros	—	Previsto no roteiro (comentado em `web.js`)

Seção 2

Visão geral da arquitetura

O sistema segue o padrão **cliente–servidor**. O navegador faz requisições HTTP ao servidor *Express*, que processa a solicitação nas camadas de roteamento, controle e acesso a dados, e devolve páginas HTML renderizadas no servidor. As folhas de estilo e os ícones são fornecidos pela CDN *jsDelivr* (Bootstrap 5.3.8 e Bootstrap Icons), sem arquivos CSS locais.



`biblioteca` web.js — rotas BookController.js — controlador Book.js + db.js — modelo (Knex) SQL (SELECT / INSERT) HTTP Recursos externos: Bootstrap 5.3.8 · Bootstrap Icons (CDN)

Figura 1 — Arquitetura geral do sistema (visão de containers).

O fluxo completo de uma requisição típica: o navegador chama uma rota registrada em `web.js`; o controlador orquestra a operação; o modelo `Book` executa a consulta/gravação via **Knex** (query builder) no MySQL; por fim, o controlador devolve a resposta (renderização de um template EJS ou mensagem de status).

Seção 3

Tecnologias e dependências

3.1 Stack principal

Tecnologia	Papel no projeto
Node.js	Runtime JavaScript que executa o servidor.
Express (v5)	Framework HTTP responsável por rotas, middleware e renderização.
EJS (v6)	Motor de templates usado para gerar o HTML no servidor.
Knex (v3)	Query builder que abstrai o SQL e gerencia a conexão.
mysql2 (v3)	Driver de comunicação com o MySQL.
Bootstrap 5.3.8	Design system via CDN (utilidades CSS e componentes JS).
Bootstrap Icons	Ícones vetoriais utilizados na interface.

3.2 Dependências do `package.json`

```
dependencies
├── ejs      ^6.0.1
├── express  ^5.2.1
├── knex     ^3.3.0
└── mysql2   ^3.23.3
```

3.3 Conexão com o banco

A conexão é criada em `src/config/db.js` através do Knex. Atualmente as credenciais estão fixadas no código (recomenda-se migrá-las para variáveis de ambiente, veja a Seção 12).

```
knex({
  client: "mysql2",
  connection: {
    host: "localhost",
    user: "admin",
    password: "****",
    database: "biblioteca"
  }
})
```

Seção 4

Estrutura de diretórios

Organização do repositório no padrão MVC:

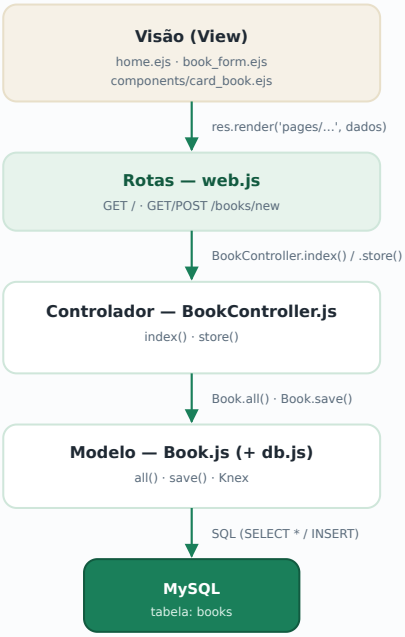
```
biblioteca/
├── index.js                    # Ponto de entrada · configuração do Express e do EJS
├── package.json                # Metadados e dependências do projeto
├── readme.md                   # Documentação de uso, instalação e configuração
│
└── src/
    ├── config/
    │   └── db.js               # Conexão com o MySQL (Knex)
    ├── controllers/
    │   └── BookController.js   # index() → lista livros · store() → cadastra livro
    ├── models/
    │   └── Book.js             # all() → SELECT · save() → INSERT
    ├── routes/
    │   └── web.js              # Registro das rotas HTTP
    └── views/
        ├── components/
        │   └── card_book.ejs   # Card individual de livro (dentro da grade)
        └── pages/
            ├── home.ejs        # Tela inicial · acervo e seções institucionais
            └── book_form.ejs   # Formulário de cadastro de livro
```

Boas práticas observadas: a separação em *rotas → controlador → modelo → visão* mantém as responsabilidades isoladas; o template de card é um componente reutilizável (`card_book.ejs`) incluído pela página inicial.

Seção 5

Arquitetura em camadas (MVC)

Cada camada tem uma responsabilidade única e se comunica apenas com a camada imediatamente subjacente, seguindo o padrão *Model-View-Controller*.



Responsabilidades por camada

✓ **Rota (Route)**
Registra os endpoints (URL → função) no Router do Express.

✓ **Controlador (Controller)**
Extrai parâmetros da requisição, chama o modelo e define a resposta.

✓ **Modelo (Model)**
Abstrai o acesso ao banco com o Knex e encapsula as regras de query.

✓ **Visão (View)**
Recebe os dados prontos e gera o HTML final (sem lógica de negócio).

✓ **Config (db.js)**
Centraliza credenciais e a instância de conexão fornecida ao modelo.

Total de responsáveis no fluxo:
1 servidor · 4 camadas · 1 banco de dados

Padrão assíncrono baseado em Promises (async/await).

Figura 2 — Camadas MVC, responsabilidades e fluxo de invocação.

Seção 6

Modelo de dados (banco de dados)

O banco contém uma única tabela, `books`. A coluna `cover` guarda a URL da imagem de capa (atribuída automaticamente pelo controlador durante o cadastro).

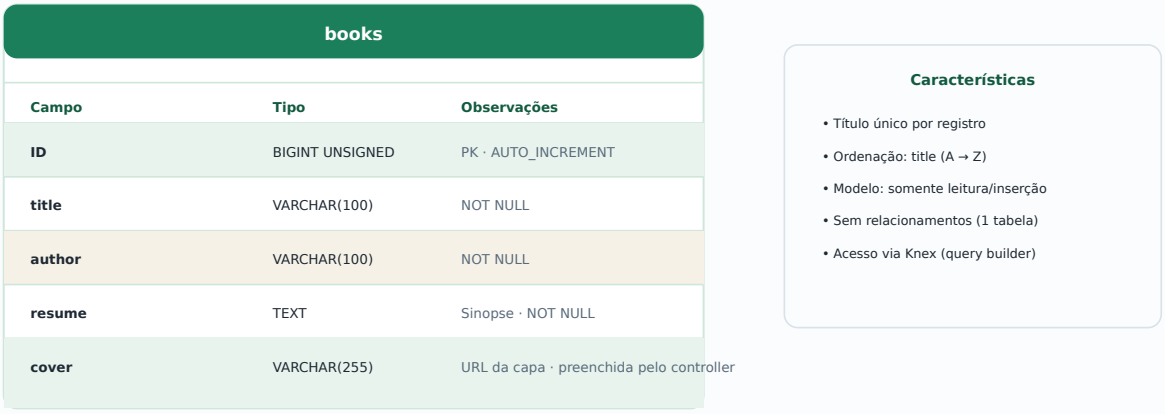


Figura 3 — Modelo entidade-relacionamento da tabela `books`.

DDL de referência

```
CREATE DATABASE biblioteca;
USE biblioteca;

CREATE TABLE books (
    ID    BIGINT UNSIGNED AUTO_INCREMENT,
    title VARCHAR(100)  NOT NULL,
    author VARCHAR(100) NOT NULL,
    resume TEXT          NOT NULL,
    cover VARCHAR(255)   NULL,
    PRIMARY KEY(ID)
);
```

Atenção: o comando acima considera a coluna `cover`. Se a base existente foi criada apenas com o `readme.md` (sem `cover`), adicione-a com `ALTER TABLE books ADD COLUMN cover VARCHAR(255) NULL;` para que o cadastro não falhe.

Seção 7

Fluxo de requisições

7.1 Listar livros — `GET /`

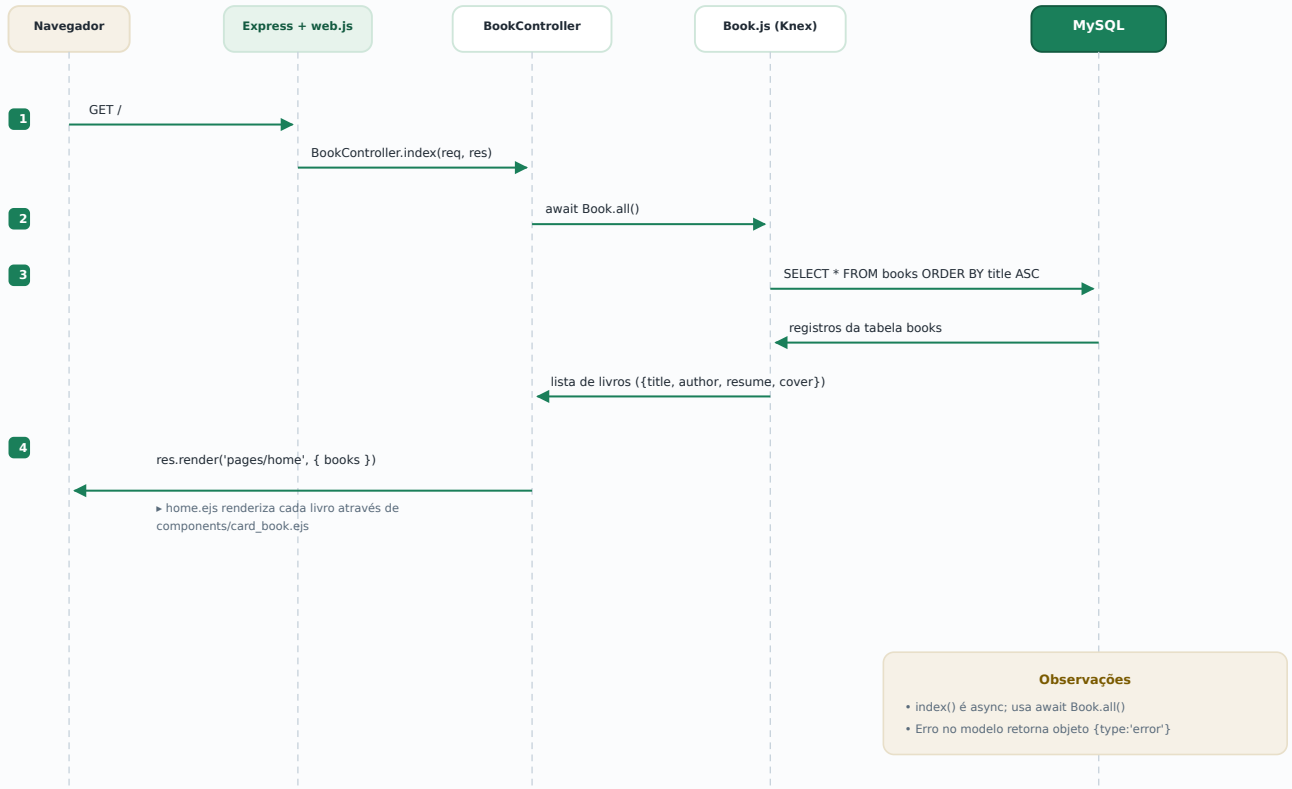


Figura 4 — Diagrama de sequência: listar o acervo (`GET /`).

7.2 Cadastrar livro — `POST /books/new`

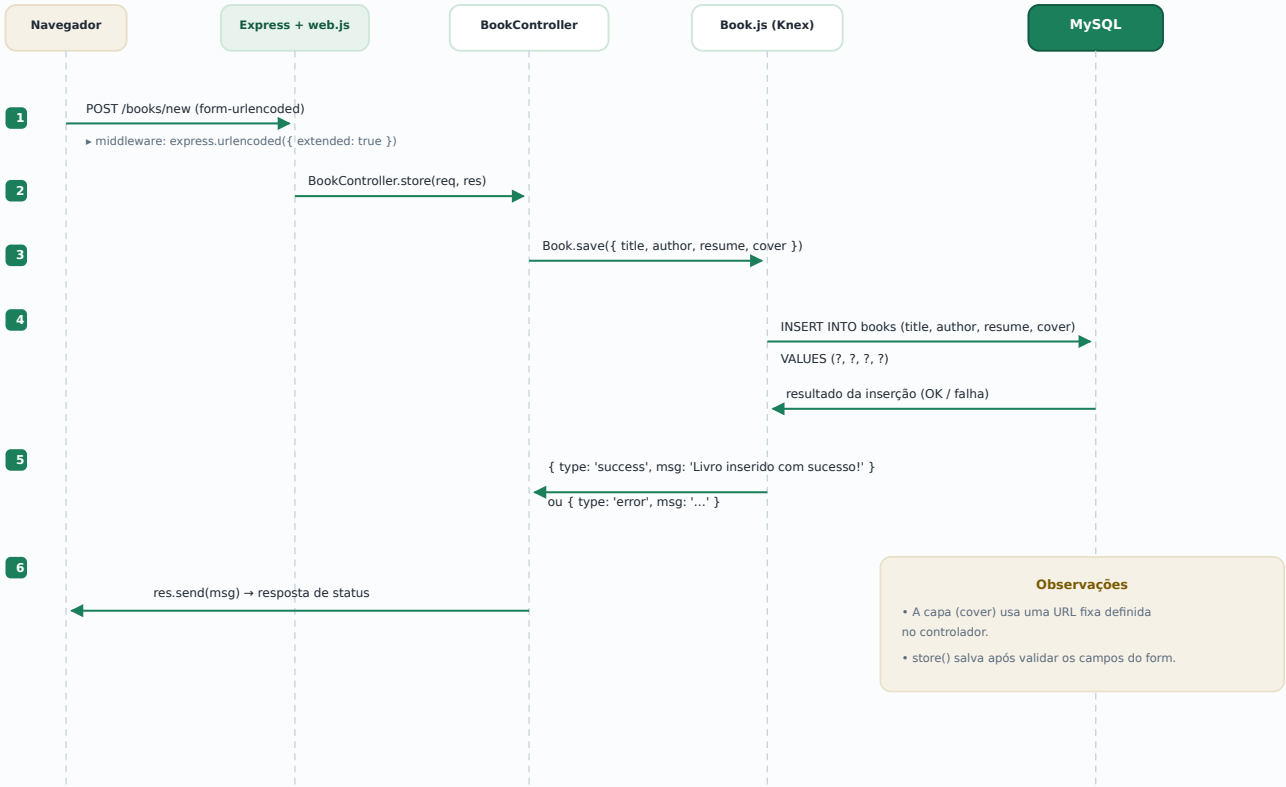


Figura 5 — Diagrama de sequência: cadastrar um livro (`POST /books/new`).

Seção 8

Rotas da aplicação

Rotas registradas em `src/routes/web.js`:

Método	Rota	Função / Tratamento	Descrição
`GET`	`/`	`BookController.index`	Consulta todos os livros e renderiza `home.ejs`.
`GET`	`/books/new`	*inline* (`res.render`)	Exibe o formulário de cadastro (`book_form.ejs`).
`POST`	`/books/new`	`BookController.store`	Insere o livro no banco e responde com o status.

Rotas previstas no roteiro

Os comentários de `web.js` indicam as próximas rotas do CRUD:

Método	Rota	Descrição
`GET`	`/books/:id`	Exibir os detalhes de um livro.
`POST`	`/books/edit/:id`	Atualizar os dados de um livro.
`POST`	`/books/delete/:id`	Excluir um livro.

Seção 9

Camada de visão (templates EJS)

Template	Rota	Conteúdo	Dados recebidos
`pages/home.ejs`	`GET /`	Navbar · hero · seções institucionais (bento grid) · grade de livros · newsletter · rodapé	`{ books }`
`pages/book_form.ejs`	`GET /books/new`	Navbar · formulário de cadastro (título, autor, sinopse) · rodapé	—
`components/card_book.ejs`	incluído por `home.ejs`	Card de livro: capa, autor, título, sinopse e botão de leitura	`{ book }`

Decisões de interface

- **Bootstrap 5.3.8** como design system, consumido via CDN;
- Tema **flat e minimalista**, em tons pastéis (verde + bege), transmitindo confiança sem peso visual;
- Tipografia grande (`display-*`), cantos arredondados (`rounded-*`), sombras suaves (`shadow-sm`) e botões pill (`rounded-pill`);
- Grade responsiva: cards adaptam-se de *4 colunas* (desktop) a *1 coluna* (celular).

Seção 10

Fluxograma geral de uma requisição

Visão consolidada do caminho percorrido por qualquer requisição:

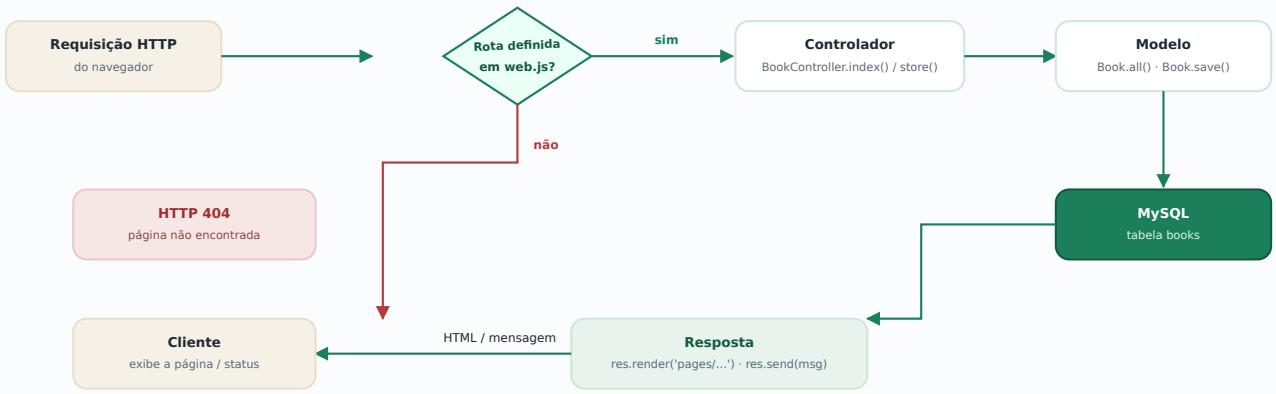


Figura 6 — Fluxograma de processamento de uma requisição HTTP.

Seção 11

Como executar o projeto

1. Instalar as dependências: `npm install`
2. Garantir o MySQL em execução e o banco `biblioteca` criado (ver Seção 6);
3. Conferir as credenciais em `src/config/db.js`;
4. Iniciar o servidor: `node index.js`
5. Acessar `http://localhost:3000`.

```
node index.js
# Rodando em: http://localhost:3000
```

Seção 12

Pontos de evolução

Item	Sugestão	Impacto
Credenciais do banco	Migrar para variáveis de ambiente (`.env`)	Segurança e portabilidade
CRUD completo	Implementar rotas de exibir / editar / excluir já comentadas em `web.js`	Completude funcional
Validação	Validar e sanitizar os campos no servidor (título/autor obrigatórios; limites de tamanho)	Integridade dos dados
Upload de capa	Salvar imagens de capa localmente ou em storage, em vez de URL fixa	Experiência de uso
Busca e paginação	Adicionar `?q=` e paginação na listagem	Escalabilidade da listagem
Testes	Incluir testes automatizados (unitários com Jest/Vitest; integração via Supertest)	Confiabilidade